USF Bellini
College of Artificial Intelligence, Cybersecurity and Computing

AI & Machine Learning Summer Intensive

Introduction to Python Programming

Dr. Kaleemunnisa

The History of Computers

From early counting tools to artificial intelligence — the journey of computing innovation.

Abacus
~3000 BCE

One of the earliest counting tools used by ancient civilizations for basic arithmetic.

Mechanical Calculator
1642

Blaise Pascal invents the Pascaline, one of the first mechanical calculators.

Analytical Engine
1837

Charles Babbage designs a general-purpose mechanical computer.

Tabulating Machine
1890

Herman Hollerith builds a machine that uses punched cards to process data.

ENIAC
1945

One of the first electronic general-purpose computers, used thousands of vacuum tubes.

Transistor Computers
1950s

Transistors replace vacuum tubes, making computers smaller, faster, and more reliable.

Integrated Circuits
1960s

Tiny chips put many transistors on a single piece of silicon, powering the computer revolution.

Personal Computers
1970s–1980s

Computers become affordable and accessible to individuals and businesses.

The Internet Era
1990s

Computers connect the world. The internet transforms communication, business, and education.

Laptops & Mobility
2000s

Powerful computers shrink in size. Laptops and mobile devices become everyday tools.

Smartphones & Apps
2010s

Computing goes mobile. Apps change the way we live, work, and connect.

Cloud Computing
2010s

Data and software move to the cloud, making computing more flexible and powerful.

Artificial Intelligence
2020s

AI and machine learning enable computers to learn, reason, and solve complex problems.

The Future
Beyond 2020s

Computing continues to evolve—driving innovation in every field and shaping a better future.

Computers Then. Limitless Possibilities Now.

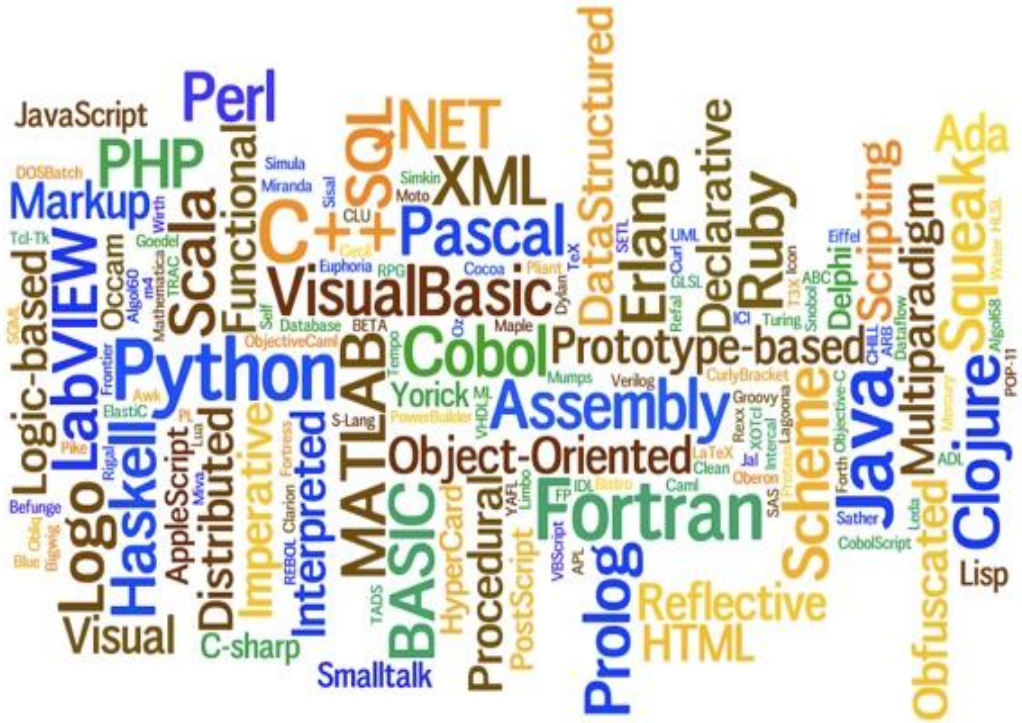
From simple counting tools to intelligent machines, the story of computers is the story of human innovation.

The best is yet to come.

Python Programming

What is Programming?

- Programming is about “instructing a computer to perform specific tasks”.
- These tasks can form an algorithm.
- A computer program consists of instructions executing one at a time.
- A programming language provides “a vocabulary and set of grammatical rules” for programming.
- Basic instruction types are:
    - Input: A program gets data.
    - Process: A program performs computations on that data.
    - Output: A program gives the computed data/results



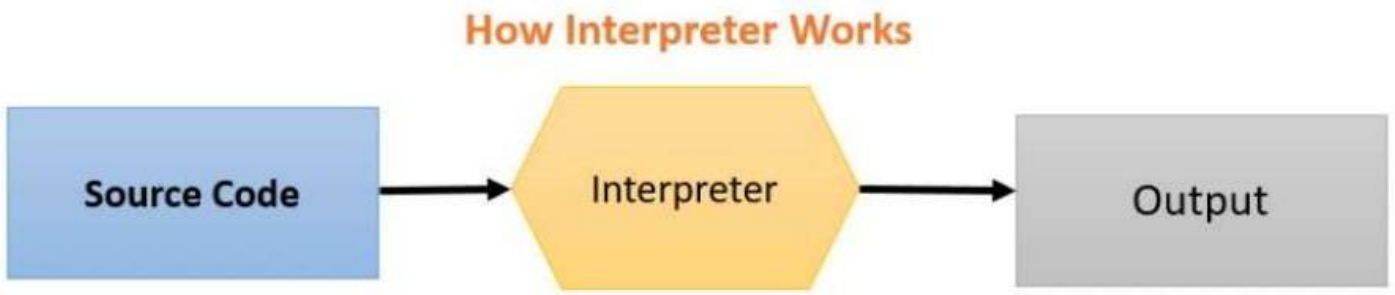
Python (https://www.python.org/)

The Python Programming Language is:

- High-level
- Interpreted – Python code runs using the Python interpreter
- Dynamically typed
- Multi-paradigm
- Known for readability, simplicity, and clarity
- Handles memory management automatically, making programming easier and more efficient
- Uses UTF-8 character encoding, so it supports many languages and symbols
- Learn more in the official Python documentation: Python Standard Library.

Python

Python is Interpreted Programming Language:



- Translates program one statement at a time.
- Interpreters usually take less amount of time to analyze the source code. However, the overall execution time is comparatively slower than compilers.
- No intermediate object code is generated, hence are memory efficient.
- Programming languages like JavaScript, Python, Ruby use interpreters.

Data Types

Data Types

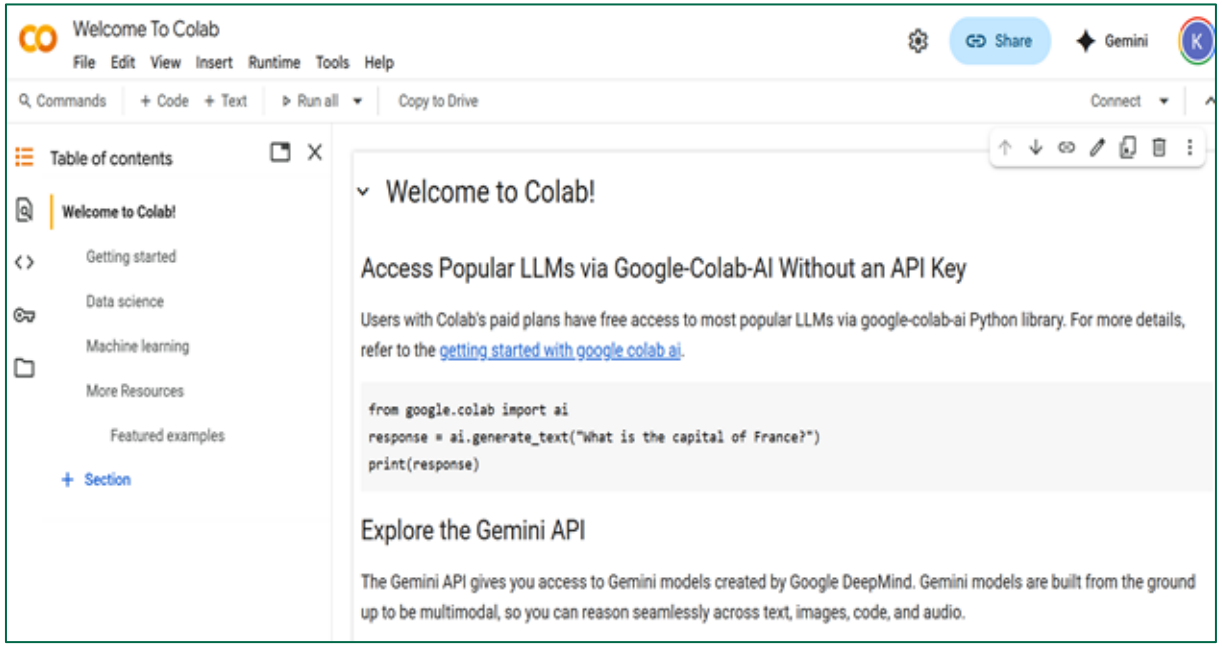
- Data Types are the building blocks in constructions of code.
- Following is the list of data types which we will be discussing in detail.

Name	Python representation	Description
Integers	int	Whole numbers, such as **1, 10, -100, 1000**
Floating point	float	Numbers with decimal points such as **1.20, 10.30 100.0**
Strings	str	Ordered sequence of characters such as **“Hello”, “$2500”**
Lists	list	Ordered sequence of objects such as **[10, “hello”, 20.5]**
Dictionaries	dict	Unordered key value pairs such as **{“Key”:”Value”, “Name”:”John”}**
Tuples	tup	Ordered sequence of objects which are immutable such as **(10, “hello”, 20.5)**
Sets	set	Unordered collection of objects which are unique **{“a”, “b”}**
Boolean	bool	Logical values indicating **True** or **False**

Google Colab

What is Google Colab

- Free, cloud-based Jupyter Notebook environment
- Run Python code without installing anything
- Comes pre-installed with popular libraries
- Provides free access to CPU, GPU, and TPU



Data Types Continue…

Printing Strings

- There are many ways to format a string object for printing. One such concept is called string interpolation. Where you can combine a string object with the text.

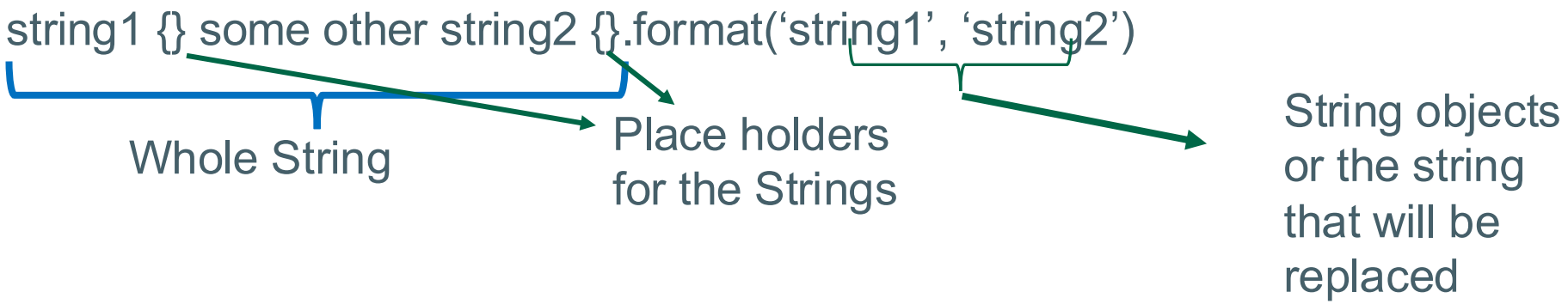
  Example : name = Johnson

  print(“Hello”+name)

- We will explore two methods:
  1. **.format()**
  2. **f-strings : Formatted string literals.**

.format()

.format() : The syntax of it is as follows




```
print("String1 {} and String2 {}".format("String1 goes here!!", "String2 goes here!!"))
```

```
String1 String1 goes here!! and String2 String2 goes here!!
```

```
string1 ="Python"
string2 = "Programming"
print("String1 {} and String2 {}".format(string1, string2))
```

```
String1 Python and String2 Programming
```

f-string

- An f-string (formatted string literal) in Python is a way to embed expressions (like variables, calculations, or function calls) directly inside a string using curly braces {}.

  Syntax: f"some text {expression} more text"
  The string must start with f or F before the quotes.

- Anything inside {} is evaluated as Python code.

```
name = “Alex"
age = 30
print(f"My name is {name} and I am {age} years old.")
```

```
x = 10
y = 5
print(f"Sum: {x + y}, Product: {x * y}")
```

Logical Operations

Logical Operations

- Python provides four sets of logical operations.
- Following is the list, which we will be discussing in detail.

Name	Description
Identity Operators	**‘is’ ,’is not’**
Comparison Operators	**== , != , < , > , >= , <=**
Membership Operator	**‘in’, ‘not in’**
Logical Operator	**and , or , not**

Control Flow Statements

Control Flow Statements

Following are the basic control flow statements.

- The “if” Statement
- The “while” Statement
- The “for” Statement
- Basic Exception Handling

The if, elif, else, Statements

- When we want, certain code be executed based on a satisfaction of condition.

  Example: if there is no class, then I will play football.

- ‘if’ Statement identifies which statement or code block to run based on the value of an expression.

- To control the flow; we use some keywords:
  - if
  - elif
  - else

Syntax : The syntax of control flow uses **‘:’** and **‘indentation’.**

Indentation ‘:’

- The indentation is very crucial to python.
- Python uses indentation for blocks, instead of curly braces.
- Both tabs and spaces are supported, but the standard indentation requires standard Python code to use four spaces.

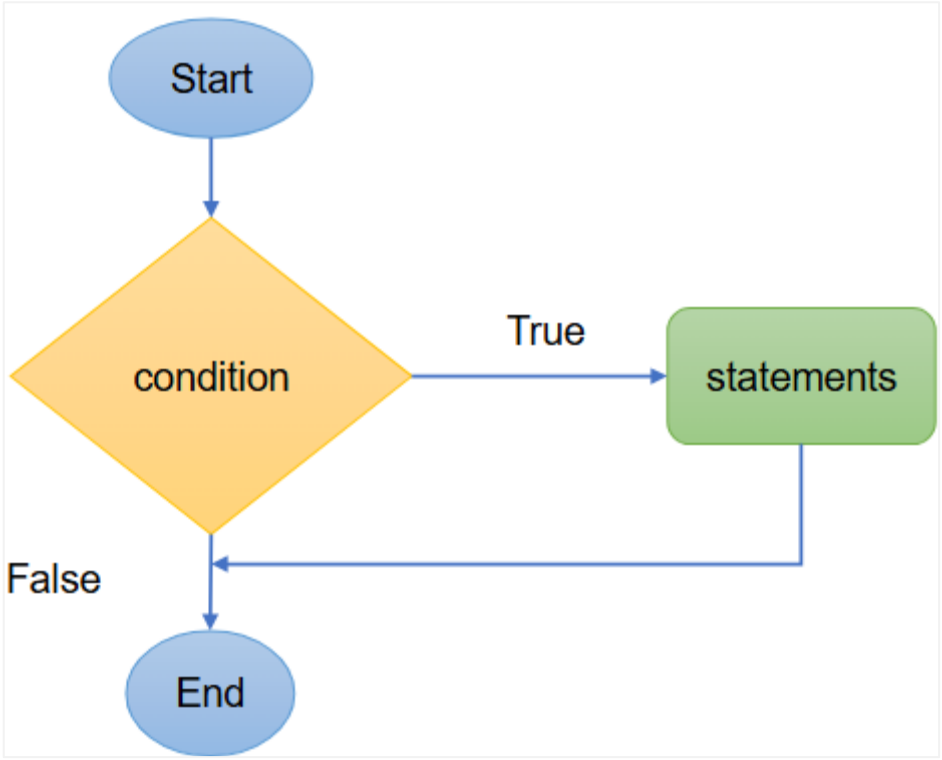
For example:

```
x = 1
if x == 1:
    print(“x is 1.”) #indented four spaces
```

- With indentation code readability is achieved and it is a core part of the design of the Python language.

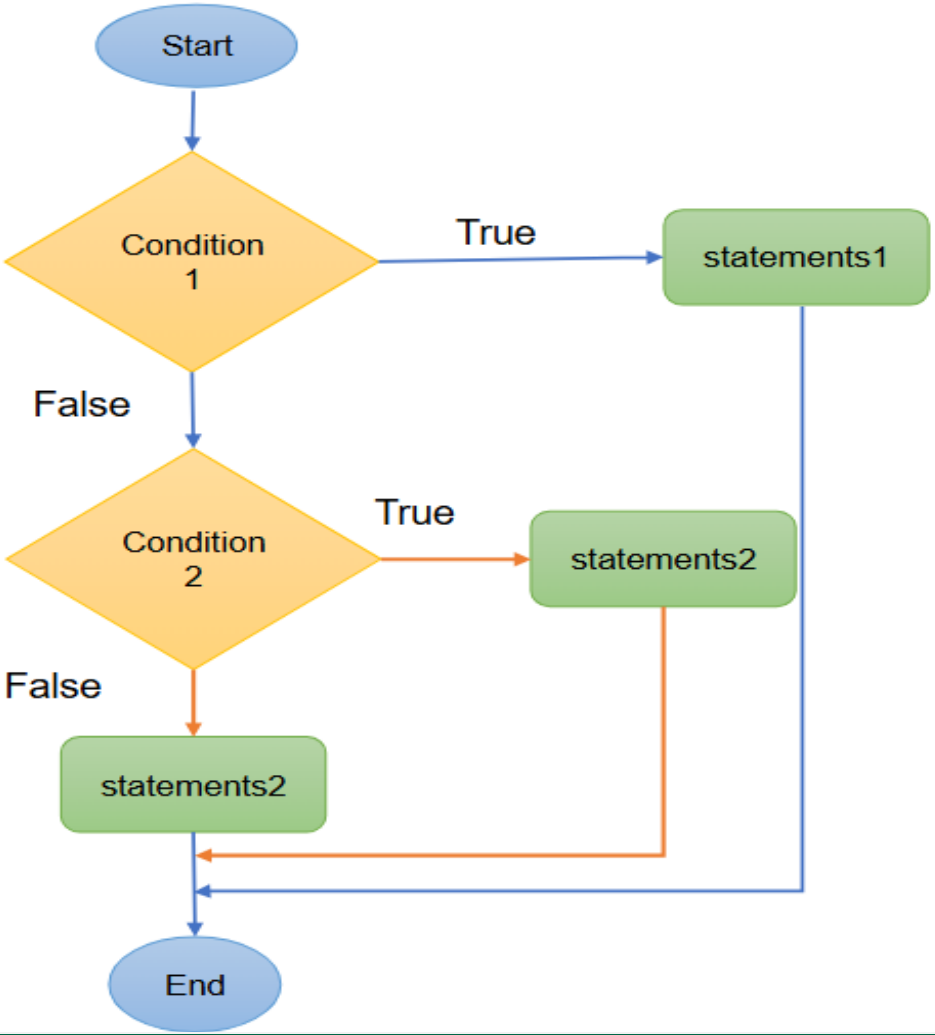
“if-else” branches

- In programming, a ***branch*** is a sequence of statements that executes only when a certain condition is met.
- The ***if statement*** makes branching possible.
- If the condition evaluates to ***True***, the program runs the statements inside the branch.
- If it evaluates to ***False***, those statements are skipped.



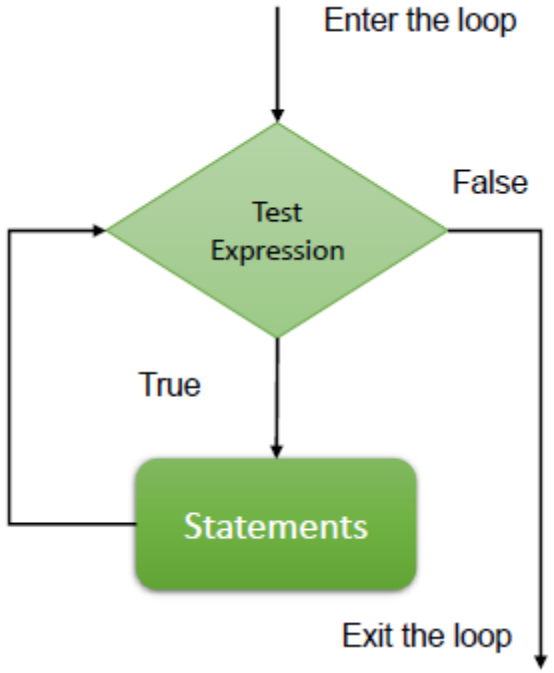
“if, elif/elseif, else” Statements

- The **if–elif–else statement** lets your program check multiple conditions in sequence.
- If the first condition is True, the corresponding block of code runs.
- If the first condition is False, the program checks the next condition with elif.
- If none of the conditions are True, the else block runs as the fallback.



Introduction to Loop

A **loop** is a programming construct that repeatedly executes a block of code as long as a specified condition remains true. Once the condition becomes false, the program continues with the next statement after the loop.



Introduction to Loops in Python

Loops are one of the most powerful and essential tools in any programming language, including Python. They allow us to automate repetitive tasks, process large datasets, and build dynamic programs with just a few lines of code.

Python supports iteration over various iterable objects such as lists, strings, and other sequence types.

There are two primary types of loops in Python:

- **for** loop - great when you know how many times to repeat.
- **while** loop - perfect when you want to repeat until a condition is no longer true

Loops in Python: “ for “ Statement

The for loop is commonly used to execute a block of code for each item in a sequence . It can be used to iterate over characters in a string, elements in a list, or keys in a dictionary. Used when the number of iterations is known.

Syntax :

```
for value in sequence :
    statements
```

Example:

```
for fruit in ['apple', 'banana', 'cherry’]:
    print(fruit)
```

range()

The **for** loop is combined with **range** method to repeat some code a certain number of times.

Example:

```
for value in range(5):
    print("hello!")
```

With **range()** method we can generate a range of numbers in a sequence.

Example :

range(10) **:** Will generate numbers from 0 to 9
Output : [0,1,2,3,4,5,6,7,8,9]

range(0, 50, 5) **:** Will generate numbers from 0 to 50 with a step of 5.
Output : [0,5,10,15,20,25,30,35,40,45]

Loops in Python: “ while “ Statement

Sometimes, you may not know in advance how many times a loop should run. Instead, you want the code to keep executing until a certain condition becomes false. In such cases, a **while** loop is the ideal choice.

Syntax :

```
while condition:
        # block of code to execute
```

Example:

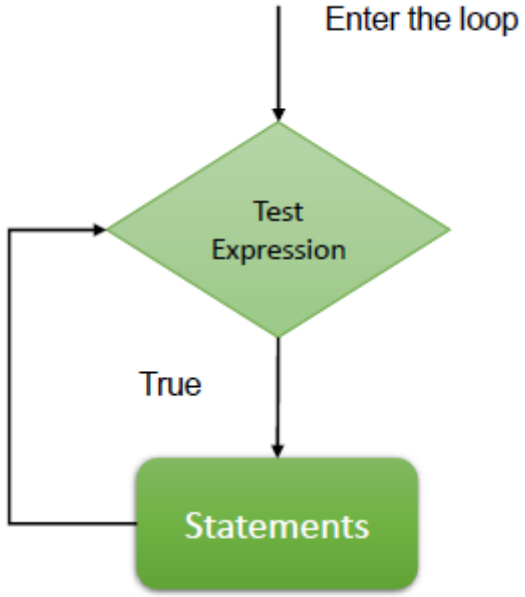
```
count = 1

while count <= 5:
    print("Count is:", count)
    count += 1
```

Infinite loop

The **infinite loop** is a special kind of while loop; it never stops running. Its condition always remains **True**.

An example of an infinite loop is as below.

```
while 1==1:
        print("In the loop")
```



Infinite Loop

Loop Control Statements

break: Exit the loop.
continue: Skip current iteration.
pass: Placeholder when no action is needed.

- '**break**' and '**continue**' are supported in loop, we can use the **'break'** statement to stop the loop even if the **while** condition is true.

- **'continue'** statement we can be used to stop the current iteration and continue with the next statement.

Common Mistakes to Avoid

Infinite loops (while True: without break) : Add a break when a condition is met

Off-by-one errors in range() : Using incorrect start or stop values in range() so the loop runs one time too many or one too few.

Modifying the list while iterating: Changing a list (e.g., removing items) while looping through it can cause **skipped elements or unexpected behavior**.

enumerate()

Normally, when you loop through a list, you only get the values. enumerate() lets you loop through both the index and the value at the same time.

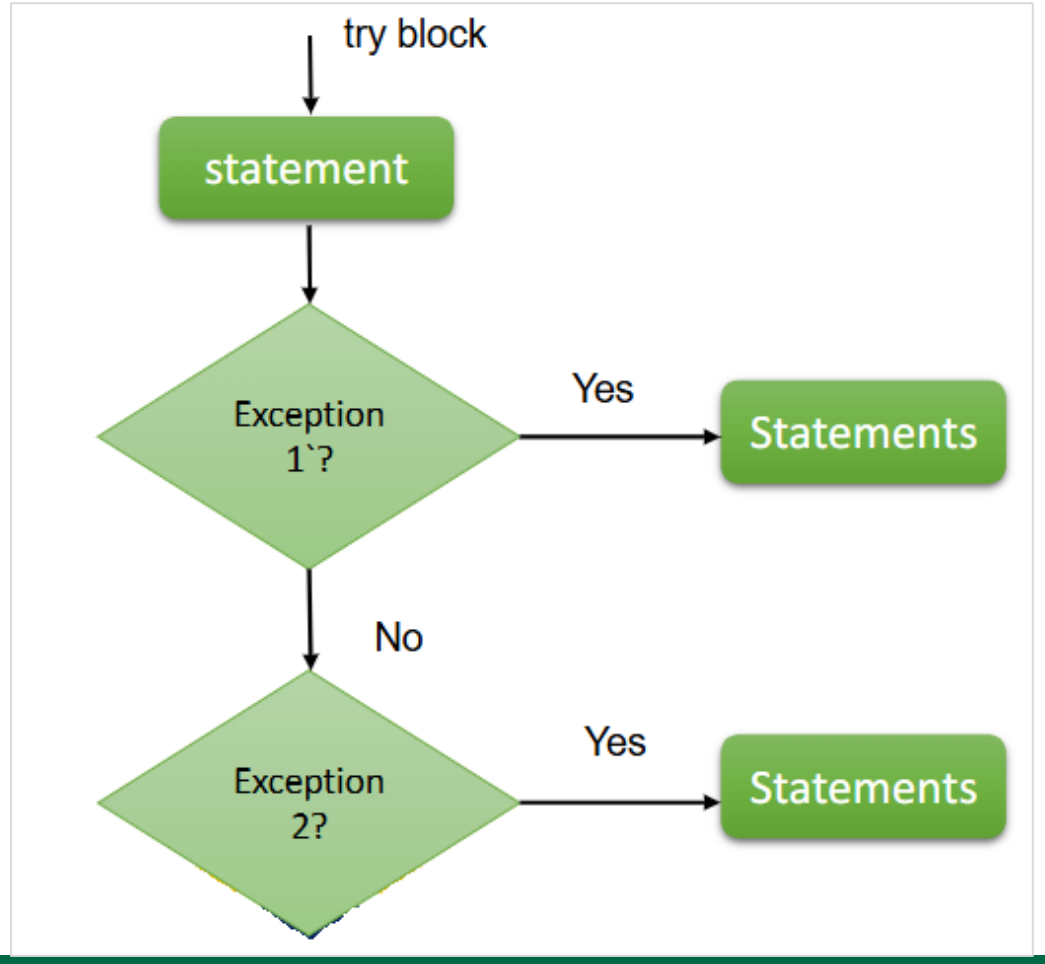
Example :

```
for i,letter in enumerate('abcde’):
        print("At index {} the letter is {}".format(i,letter))


fruits = ["apple", "banana", "cherry"]

for index, fruit in enumerate(fruits):
    print(index, fruit)
```

Basic Exception Handling

- An Exception occur when something goes wrong, due to incorrect code or input. When an exception occurs, the program immediately stops
- Exceptions provide a useful means of changing the flow of control
- A simple form of the syntax for exception handlers is this:

```
try:
    try_suite
except exception1 as variable1:
    exception_suite1
…
except exceptionN as variableN:
    exception_suiteN
```



Functions

Methods and Functions

Four kinds of function we can create in Python.

- Global functions
- Local functions
- Lambda functions
- Methods

Global functions/ Objects are accessible to any code within the same file in which it is created as well as you can call it in another file or module.

- Local functions: They are declared within other function. They are accessible with the function where it is created. They are created only for a specific function and have no use anywhere else.

- Lambda functions are created for limited use.

- Methods are functions which are associated with a particular object.

Methods

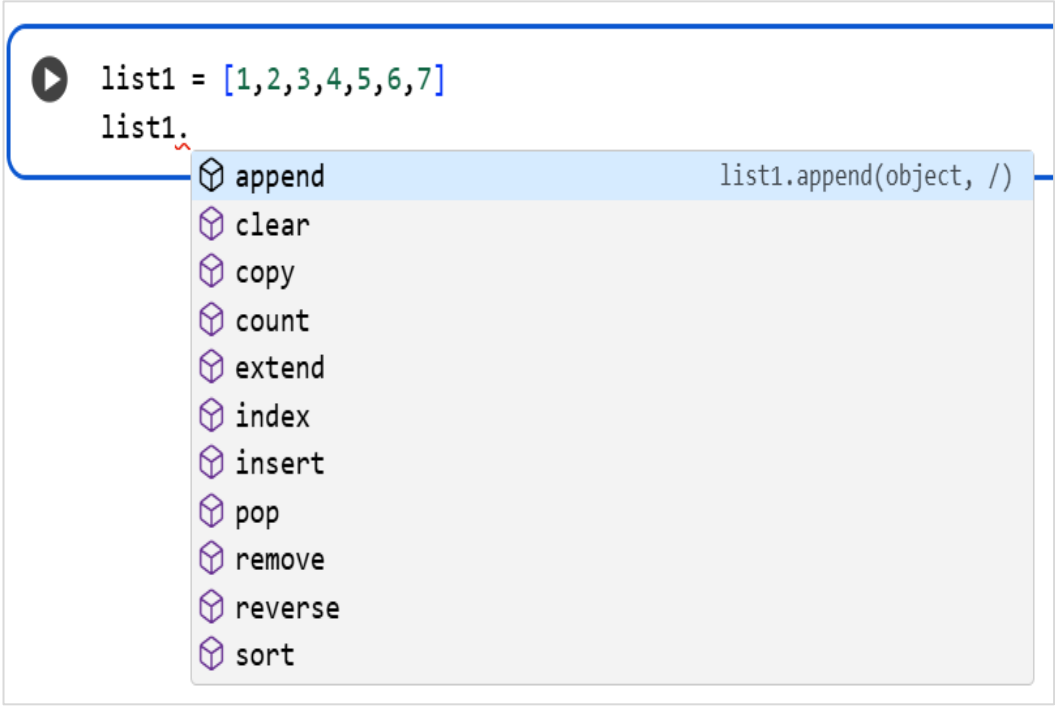
Methods are essential functions built into objects; in python they have a variety of methods you can use.

Methods perform specific actions on an object and can also take arguments, just like a function.

Syntax : object.method(arg1, arg2, arg3...)

In the example below, we can see the methods associated with list object.

Python provides many built-in methods, and the standard library and third party libraries add hundreds more (thousands if we count all the methods), so in many cases the functionality we want has already been written.



Functions

- It is a **block** of code. A **block** is a **list of statements**.
- Is a better way of ensuring that your code is not duplicated and makes your code a lot easier to maintain.
- To reduce redundancy, when you must preform a repeated task again and again.
- Can prevent a main program from becoming large and confusing.
- The general syntax for creating a (global or local) customized function is:

```
def functionName(parameters):
    ‘’’ Doc string explaining the function’’’
      Statements
```

We begin with keyword **‘def ’**, then a space followed by the **name of the function**. Name has to be relevant to its functionality, for example, if you are writing a function of adding numbers you may use add(), is a good name. Unlike variables you can not call a function the same name as a built-in function in Python.

Functions

Naming convention for functions:

- UPPERCASE for constants
- TitleCase for classes (including exceptions)
- camel-Case for GUI (Graphical User Interface) functions
- lowercase or lowercase_with_underscores for everything else
- Avoid abbreviations
- Be proportional with variable and parameter names.
- Functions should have names that say what they do.

Simple example of a function:
Defining a function:

```
def say_hello():
    print('hello’)
```

Calling a function:

```
say_hello()
Output : hello
```

Function with parameter

Simple example of a function with parameter

Defining a function:

```
def say_hello(name):
    print('Hello' +name)
```

Calling a function:

```
say_hello(' Aaron')
Output : Hello Aaron
```

‘return’

- In functions we use ‘return’ keyword to send back the result of the function, instead of printing it.
- ‘return’ allows us to assign the output of the function to new ariable.
- Example: In this example function is converting the value into an integer and returns the value to a variable.

```
def convert_to_int(value):
  """Converts a value to an integer and returns it."""
  return int(value)

#Example usage:
my_value = "123"
integer_value = convert_to_int(my_value)
print(integer_value)
```

```
123
```

Lambda Expressions, map, and filter

Map :

map(func, *iterables) - Makes an iterator that computes the function using arguments from each of the iterable.

Example :

```
# A simple function to double a number
def double(n):
  return n * 2

# A list of numbers
numbers = [1, 2, 3, 4, 5]

# Use map() to apply the double function to each element in the list
doubled_numbers = map(double, numbers)

# Convert the map object to a list to display the result
print(list(doubled_numbers))
```

```
[2, 4, 6, 8, 10]
```

Lambda Expressions, map, and filter

Filter :

filter(function, *iterable) - The filter() method filters the given sequence of iterable elements which can be sets, lists, tuples, etc, with the help of a function that tests each element in the sequence to be true or not.

Example :

```
# A simple function to check if a number is even
def is_even(n):
  return n % 2 == 0

# A list of numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Use filter() to get only the even numbers from the list
even_numbers = filter(is_even, numbers)

# Convert the filter object to a list to display the result
print(list(even_numbers))
```

```
[2, 4, 6, 8, 10]
```

Lambda Expressions, map, and filter

Lambda:

Lambda expressions are the way to quickly create anonymous functions. Or the functions which we basically use only once.

lambda parameters : expression

To define a lambda expression, you do not have to use the keyword ‘def’ or ‘function name’.

Example :

```
# A lambda function that adds 10 to a number
add_ten = lambda x: x + 10

# Example usage:
result = add_ten(5)
print(result)
```

```
15
```

Lambda Expressions, map, and filter

Lambda:

Lambda expressions are the way to quickly create anonymous functions. Or the functions which we basically use only once.

lambda parameters : expression

To define a lambda expression, you do not have to use the keyword ‘def’ or ‘function name’.

Example :

```
# A lambda function that adds 10 to a number
add_ten = lambda x: x + 10

# Example usage:
result = add_ten(5)
print(result)
```

```
15
```

Assertions

The assert statement tests for conditions in Python and it is used for debugging to handle the errors.

Syntax :

assert boolean_expression, optional_expression

```
def divide(a, b):
  # Assert that the denominator is not zero
  assert b != 0, "Division by zero is not allowed"
  return a / b

# Example usage:
result1 = divide(10, 2)
print(f"10 divided by 2 is: {result1}")

# This will raise an AssertionError
try:
  result2 = divide(10, 0)
  print(f"10 divided by 0 is: {result2}")
except AssertionError as e:
  print(f"Error: {e}")
```

```
10 divided by 2 is: 5.0
Error: Division by zero is not allowed
```

OS Module

os Module

- The os module allows Python to interact with the operating system.
- This module provides a portable way of using operating system dependent functionality.
- It provides functions for:
  - File and directory handling
  - Environment variables
  - Process management
  - Path manipulation
- It is part of the Python Standard Library (no installation needed).

Importing the OS Module

import os **->** Once imported, you can access all its functions.

```
print(os.name)  # Returns 'posix' (Linux/macOS) or 'nt' (Windows)
```

Working with Directories

- os.getcwd() # Get current working directory
- os.chdir('path')# Change directory
- os.listdir() # List all files and folders
- os.mkdir("new_folder") # Create single directory
- os.makedirs("a/b/c") # Create nested directories
- os.rmdir("new_folder") # Remove single directory
- os.removedirs("a/b/c") # Remove nested directories

Pandas

Pandas

- Pandas is an open-source library that provides high performance easy to use data analysis and data manipulation tool.
- It is built on top of NumPy.
- Pandas has built-in visualization features.
- Pandas can work with data from a wide variety of sources.

To install pandas:

```
conda install pandas
```

or

```
pip install pandas
```

[pandas] is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods for the same individuals. — Wikipedia

Pandas

When working with datasets we need a method to group data of different types. (e.g., categorical, continuous, time series .) This can be achieved using Pandas.

import pandas as pd

Pandas has following fundamental data structures:

Series: A one-dimensional array like structure with homogeneous data. Key Points: - Homogeneous data – Size Immutable - Values of Data Mutable.

DataFrame: A two-dimensional array with heterogeneous data.

Series

- A **Pandas Series** is like a **one-column table** — it's a one-dimensional list of data with labels (called *indexes*).
- You can create a Series from a **Python list** or a **NumPy array**.
- A Series has two main parts:
  - **Values:** the actual data.
  - **Index:** the labels that help you identify each value.
- Think of it like an improved **NumPy array**. While NumPy uses numbers (0, 1, 2, …) as indexes automatically, Pandas lets you **set your own labels** for the data.

```
ser1 = pd.Series(list1)
```

```
ser1
```

```
0    100
1    200
2    300
3    400
4    500
5    600
6    700
dtype: int64
```

Series

```
import pandas as pd
import numpy as np
```

Importing libraries

```
list1 =[1,2,3,4,6,7]
list1[2]
```

```
3
```

```
#Creating a series
ser1 = pd.Series(list1)
ser1
```

Create a series form a list.

```
0    1
1    2
2    3
3    4
4    6
5    7
dtype: int64
```

```
list2 =[0.25, 0.5, 0.75, 1.0]
ser2 = pd.Series(list2, index=['a','b','c','d'])
ser2
```

In series you can *explicitly defined* index associated with the values.

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```

Series

```
ser2.values
```

```
array([0.25, 0.5 , 0.75, 1.  ])
```

To get only values from a series.

```
ser2.index
```

```
Index(['a', 'b', 'c', 'd'], dtype='object')
```

To get only index from a series.

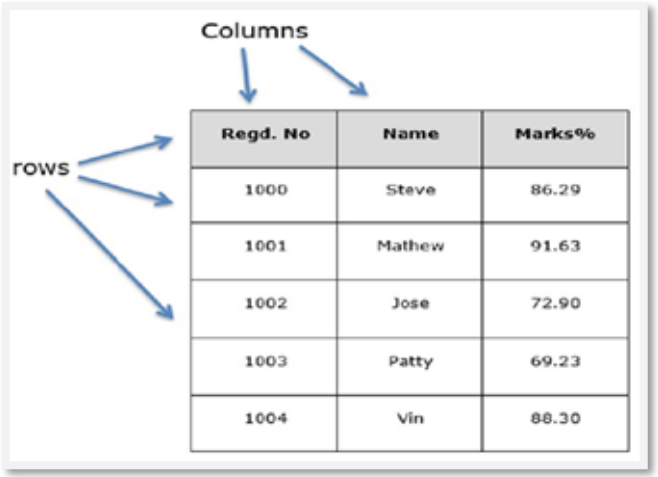
```
ser2[0]
```

```
0.25
```

To extract the value based on index from a series.

Data Frame

- A Pandas DataFrame is like a table — it has rows and columns, just like an Excel spreadsheet.
- You can think of it as a 2D version of a Series or as a dictionary of Series, where each column has a name and holds data.
- A DataFrame has flexible row indexes and column names, so you can easily organize and access your data.
- Pandas can load data from many sources such as CSV files, Excel sheets, JSON files, HTML tables, SQL databases, and more.
- When you load data using Pandas, it's stored in a structure called a DataFrame (DF) — which makes data analysis and manipulation in Python much easier.



Missing Data in Pandas

Handling Missing Data:

- In most datasets, some values will be missing.
- Missing data is often shown as null, NaN (Not a Number), or NA in Pandas.
- Ways to Represent Missing Data:
  There are a few common methods used to mark missing data in tables or DataFrames — each with its own pros and cons:

Useful Pandas DataFrame Operations

Creating and Loading Data

Function	Description
pd.DataFrame()	Create a new DataFrame manually
pd.read_csv('file.csv')	Load data from a CSV file
pd.read_excel('file.xlsx')	Load data from an Excel file
pd.read_json('file.json')	Load data from a JSON file
pd.read_sql(query, conn)	Load data from an SQL database
df.to_csv('file.csv')	Save DataFrame to a CSV file
df.to_excel('file.xlsx')	Save DataFrame to an Excel file

Useful Pandas DataFrame Operations

Exploring Data

Function	Description
df.head(n)	Show first *n* rows (default 5)
df.tail(n)	Show last *n* rows
df.shape	Get number of rows and columns
df.info()	Get info about columns, data types, and nulls
df.describe()	Summary statistics for numeric columns
df.dtypes	Show data types of columns
df.columns	List all column names
df.index	Show index (row labels)
df.sample(n)	Randomly select *n* rows

Useful Pandas DataFrame Operations

Selecting and Filtering

Function	Description
df['col']	Select a single column
df[['col1', 'col2']]	Select multiple columns
df.loc[row_label, col_label]	Access data by **label**
df.iloc[row_index, col_index]	Access data by **position**
df[df['col'] > value]	Filter rows based on a condition
df.query('col > value')	Filter using a query string

Useful Pandas DataFrame Operations

Summary and Aggregation

Function	Description
df.sum()	Sum of values
df.mean()	Mean of values
df.median()	Median of values
df.min() / df.max()	Minimum / maximum values
df.count()	Count of non-null values
df.std()	Standard deviation
df.var()	Variance
df.corr()	Correlation between columns

Useful Pandas DataFrame Operations

Handling Missing Data

Function	Description
df.isnull()	Detect missing values (returns True/False)
df.notnull()	Detect non-missing values
df.dropna()	Remove rows with missing values
df.fillna(value)	Replace missing values with a given value
df.replace(old, new)	Replace specific values

Useful Pandas DataFrame Operations

Exporting Data

Function	Description
df.to_csv()	Save to CSV
df.to_excel()	Save to Excel
df.to_json()	Save to JSON
df.to_sql()	Save to SQL database

Visualizations

Data visualization is concept of placing the data in a visual context.

With the help of Data Visualization, patterns, trends and correlations can be exposed. It enables stakeholders and decision makers to analyze data visually.

Benefits Include:

- Simplify complex Data
- Explore Big Data
- Relations between Features
- Area of improvement
- Explore Patterns

Visualizations

- Histogram : A histogram is a graphical display of data using bars of different heights. In a histogram, each bar groups numbers into ranges. Taller bars show that more data falls in that range. A histogram displays the shape and spread of continuous sample data.
- Column Chart: Comparing values, example week over week, month over month, etc
- Box plot chart : A box plot is a graphical representation of statistical data based on the minimum, first quartile, median, third quartile, and maximum.
- Line chart : To demonstrate the change in values of one variable for continuous value of another variable.
- Pie Chart : A pie in a pie chart represents a category of data, that makes a 100% of whole data.
- Scatter plot : Represents the relationship between two variables.
- Violin plot : Compare the distribution of a numeric variable across levels of a categorical variable.

Data Visualizations - Pandas

Pandas has built-in plotting functions that make it easy to visualize your data directly from a DataFrame or Series:

- df['col'].plot.area() – Shows how data changes over time with filled areas under the curve.
- df['col'].plot.bar() – Compares categories using rectangular bars.
- df['col'].plot.line() – Displays data points connected by a line, useful for showing trends.
- df['col'].plot.scatter(x, y) – Uses Cartesian coordinates (x, y) to show the relationship between two numeric variables.

Data Visualizations - Pandas

- df['col'].plot.box() – Shows the distribution of numerical data using quartiles (helps spot outliers).
- df['col'].plot.hexbin(x, y) – Similar to a scatter plot but groups data into hexagonal bins; great for large datasets.
- df['col'].plot.kde() – Draws a smooth curve showing the probability density of the data.
- df['col'].plot.density() – Another way to display the distribution of numerical values (same as kde).

QUESTIONS?